



UNIVERSITY  
OF HULL

# Win32 synchronisation & locks

# Mutual Exclusion

- Spinlock
  - Very lightweight
  - Requires spin locks that can be costly
  - Mutual exclusion across multiple threads within a single process
- Critical section
  - Lightweight, and the simplest to use
  - Mutual exclusion across multiple threads within a single process
- Slim Reader/Writer
  - Lightweight
  - Mutual exclusion across multiple threads within a single process
- Mutex
  - Critical sections, but work across processes
  - More expensive than critical sections
  - Provides both mutual exclusion
- Semaphore
  - Non-binary version of the mutex
  - Provides both mutual exclusion
- See Platform SDK: DLLs, Processes, and Threads online help

# Spinlocks

- Provides mutual exclusion

```
1. // Initialize
2. const long open=1, blocked=2;
3. long flag = 1;
4. ...
5. // Control access
6. while (InterlockedCompareExchange(&flag, blocked, open) == blocked)
7.     Sleep(1000);
8. // access data
9. InterlockedExchange(&flag, open) ;
10....
```

- Other functions available

- 80+ functions

# Critical Sections

- Provides mutual exclusion

```
1. // Initialize
2. CRITICAL_SECTION cs;
3. InitializeCriticalSection(&cs);
4. ...
5. // Control access
6. EnterCriticalSection(&cs);
7. // access data
8. LeaveCriticalSection(&cs);
9. ...
10. // Destroy
11. DeleteCriticalSection(&cs);
```

- Other functions available

- TryCriticalSection, InitializeCriticalSectionAndSpinCount

# Slim Reader/Writer lock (SRW)

- Provides mutual exclusion

```
1. // Initialize
2. SRWLOCK rwLock;
3. InitializeSRWLock(& rwLock);
4. ...
5. // Control access
6. AcquireSRWLockExclusive(& rwLock);
7. // access data
8. ReleaseSRWLockExclusive(& rwLock);
9. ...
10. // Destroy
11. DeleteCriticalSection(& rwLock);
```

- Other functions available

- AcquireSRWLockShared, AcquireSRWLockShared

# Mutexes

- Synchronise data access across multiple processes

```
1. // Initialize
2. HANDLE  hMutex = CreateMutex (NULL, FALSE, NULL);
3. ...
4. // Control access
5. if (WaitForSingleObject(hMutex, INFINITE) != WAIT_OBJECT_0)
6.     return;
7. // access data
8. ReleaseMutex(hMutex);
9. ...
10. // Destroy
11. CloseHandle(hMutex );
```

- Other functions available

- OpenMutex

# Semaphores

- Synchronise data, but permitting multiple threads to have access

- `// Initialize`
- `HANDLE hSem = CreateSemaphore( NULL, 0, 2, NULL);`
- `...`
- `// Control access`
- `if (WaitForSingleObject(hSem, INFINITE) != WAIT_OBJECT_0)`
- `return;`
- `// access data by TWO threads concurrently`
- `ReleaseSemaphore(hSem, 1, NULL);`
- `...`
- `// Destroy`
- `CloseHandle(hSem );`

- Other functions available

- `OpenSemaphore`

# Synchronisation

- Events
  - For signalling the occurrence of an event
  - Works across processes
- Condition variables
  - Lightweight signals for use on a single process
  - Can be used in conjunction with critical sections and SRWlocks
- Mutex/Semaphores
- See Platform SDK: DLLs, Processes, and Threads online help



# Events

- Signal an event across multiple processes

```
1. // Initialize
2. HANDLE hEvent= CreateEvent (NULL, TRUE, FALSE, NULL);
3. ...
4. // Wait for an event
5. if (WaitForSingleObject(hEvent, INFINITE) != WAIT_OBJECT_0)
6.     return;
7. ResetEvent(hEvent);

8. // Separate thread
9. SetEvent(hEvent);

10. // Destroy
11. CloseHandle(hEvent);
```

- Other functions available

- OpenEvent, PulseEvent

# Condition variables

```
1. // Initialize
2. CRITICAL_SECTION cs;
3. CONDITION_VARIABLE cv;
4. InitializeCriticalSection(&cs);
5. ...
6. // Control access
7. EnterCriticalSection(&cs);
8. SleepConditionVariableCS(&cv, &cs, INFINITE)
9. // access data
10. LeaveCriticalSection(&cs);

11. //Separate thread
12. WakeConditionVariable(&cv);

13. // Destroy
14. DeleteCriticalSection(&cs);
```

- Other functions available: WakeAllConditionVariable



UNIVERSITY  
OF HULL

# OpenMP part 2

Synchronization and critical sections

# Example: Numerical Integration

```
1. void main(int, char**) {  
2.     double area=0.0;  
3.     const int limit = 100000;  
  
4.     for (int i=0; i<limit; i++) {  
5.         double x = (i+0.5) / limit;  
6.         area += 4.0/(1.0 + x*x);  
7.     }  
  
8.     double pi = area / limit;  
9.     cout << "PI= " << pi << endl;  
10. }
```

# Example: Numerical Integration (with OpenMP)

```
1. void main(int, char**) {  
2.     double area=0.0;  
3.     const int limit = 100000;  
  
4.     #pragma omp parallel for  
5.     for (int i=0; i<limit; i++) {  
6.         double x = (i+0.5) / limit;  
7.         area += 4.0/(1.0 + x*x);  
8.     }  
  
9.     double pi = area / limit;  
10.    cout << "PI= " << pi << endl;  
11. }
```

# Example: Numerical Integration (with OpenMP)

```
1. void main(int, char**) {  
2.     double area=0.0;  
3.     const int limit = 100000;  
  
4.     #pragma omp parallel for  
5.     for (int i=0; i<limit; i++) {  
6.         double x = (i+0.5) / limit;  
7.         #pragma omp critical  
8.         area += 4.0/(1.0 + x*x);  
9.     }  
  
10.    double pi = area / limit;  
11.    cout << "PI= " << pi << endl;  
12.}
```

# Reductions

- Given  $n$  variables  $a_0, a_1 \dots a_n$ , reduction is the process of calculating  $a_0 * a_1 * \dots * a_n$ , where  $*$  is an operator.
- OpenMP statement specifies
  - Reduction variable
  - Reduction operator
  - `#pragma omp parallel for reduction(+:area)`

# Example: Numerical Integration (with OpenMP)

```
1. void main(int, char**) {  
2.     double area=0.0;  
3.     const int limit = 100000;  
  
4.     #pragma omp parallel for reduction(+:area)  
5.     for (int i=0; i<limit; i++) {  
6.         double x = (i+0.5) / limit;  
7.         area += 4.0/(1.0 + x*x);  
8.     }  
  
9.     double pi = area / limit;  
10.    cout << "PI= " << pi << endl;  
11. }
```



# Synchronization Example

1. `#pragma omp parallel`
2. `{`
3.     `int id = omp_get_thread_num();`
4.     `cout << "Thread" << id << endl;`
5. `}`

# OMP Synchronization APIs

| OpenMP name      | Wraps Windows:            |
|------------------|---------------------------|
| omp_lock_t       | CRITICAL_SECTION          |
| omp_init_lock    | InitializeCriticalSection |
| omp_destroy_lock | DeleteCriticalSection     |
| omp_set_lock     | EnterCriticalSection      |
| omp_unset_lock   | LeaveCriticalSection      |
| omp_test_lock    | TryEnterCriticalSection   |

# Synchronization Example

```
1. omp_lock_t lk;  
2. omp_init_lock( &lk );  
3. #pragma omp parallel  
4. {  
5.     int id = omp_get_thread_num();  
6.     omp_set_lock( &lk );  
7.     cout << "Thread" << id << endl;  
8.     omp_unset_lock( &lk );  
9. }  
10. omp_destroy_lock( &lk );
```

# Nested loops

- Simple nested loops can be parallelized using parallel for pragmas on the outer loop

```
1. #pragma omp parallel for
2. for (int i=0; i<limit; i++) {
3.     for (int j=0; j<limit; j++) {
4.     }
5. }
```

# Nested loops – with break

- Nested loops containing breaks cannot be simply parallelized

```
1. #pragma omp parallel for
2. for (int i=0; i<limit; i++) {
3.     if (flag) break;
4.     for (int j=0; j<limit; j++) {
5.     }
6. }
```

■ Invalid

```
1. for (int i=0; i<limit; i++) {
2.     if (flag) break;
3.     #pragma omp parallel for
4.     for (int j=0; j<limit; j++) {
5.     }
6. }
```

■ Inefficient

# Nested loops – with break

- The outer loop is parallelized, but all threads execute all the code
- The inner loop is parallelized, but each iteration is assigned to only one thread.

```
1. #pragma omp parallel
2. for (int i=0; i<limit; i++) {
3.     if (flag) break;
4.     #pragma omp for
5.     for (int j=0; j<limit; j++) {
6.     }
7. }
```

# Nested loops – single statement

```
1. #pragma omp parallel
2. for (int i=0; i<limit; i++) {
3.     cout << i << endl;
4.     if (flag) break;
5.     #pragma omp for
6.     for (int j=0; j<limit; j++)
7.         {...}
8. }
```

```
1. #pragma omp parallel
2. for (int i=0; i<limit; i++) {
3.     #pragma omp single
4.     cout << i << endl;
5.     if (flag) break;
6.     #pragma omp for
7.     for (int j=0; j<limit; j++)
8.         {...}
9. }
```

- How many prints ?

# Synchronization of iterations

- The Parallel For statement, by default has a synchronization barrier after each iteration
  - All threads must complete iteration N, before any can start iteration N+1
- Nowait: removes synchronization barrier
- Looping over a linked list:

```
1. #pragma omp parallel
2. for( p = list; p != NULL; p = p->next )
3.     #pragma omp single nowait
4.     process( p ); // efficient if processor intensive
```



# Functional parallelism

- Executing concurrent functions

```
1. #pragma omp parallel sections
2. {
3.     #pragma omp section
4.     Xaxis();
5.     #pragma omp section
6.     Yaxis();
7.     #pragma omp section
8.     Zaxis();
9. }
```



UNIVERSITY  
OF HULL

# Thread Models

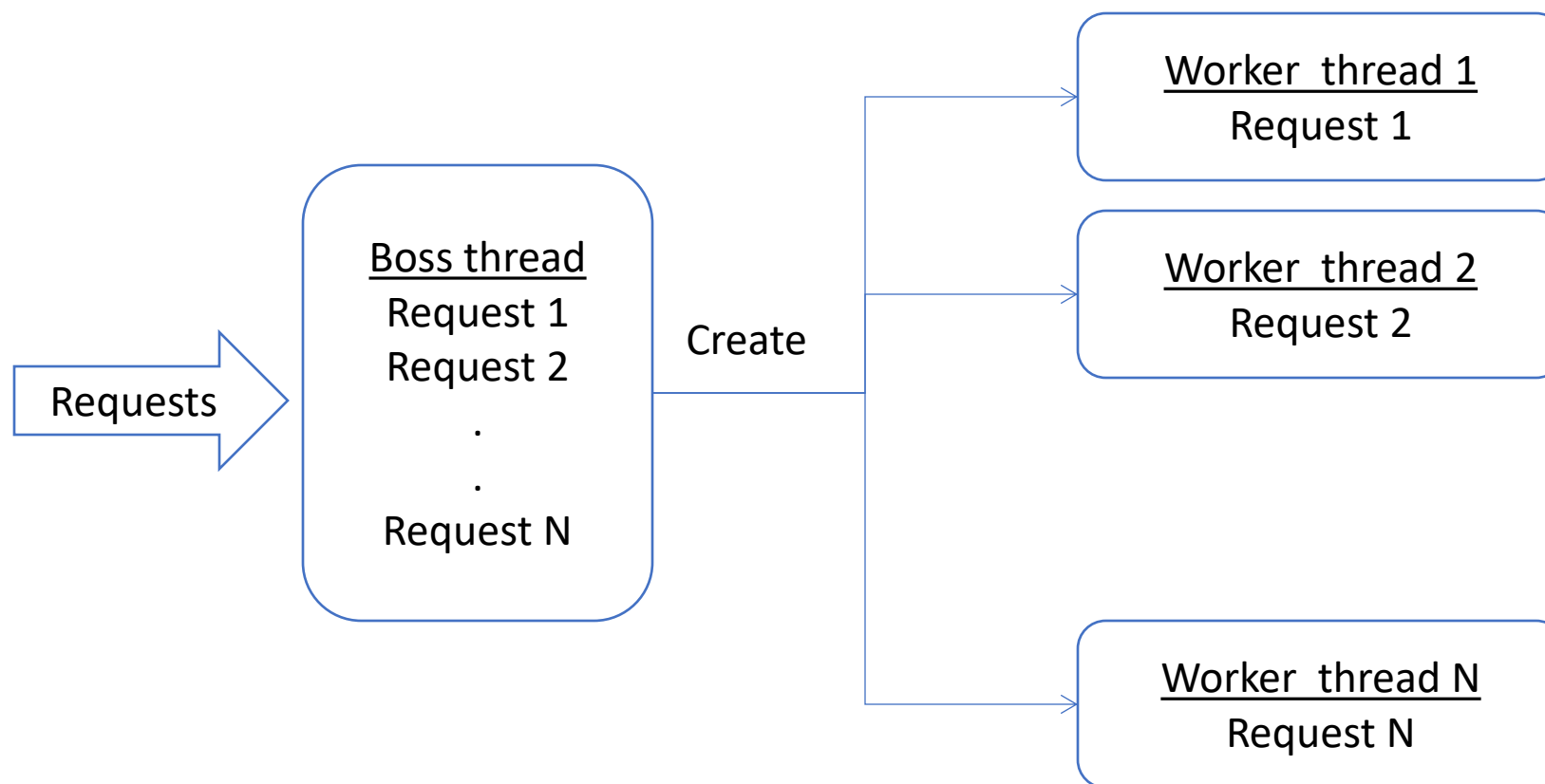
# Thread models

- Delegation
- Peer-to-peer
- Pipeline
- Producer-consumer

# Delegation model

- Single “boss” thread
  - Creates worker threads on demand
  - Assigns tasks to worker threads
  - Can be considered an event handler
- Considerations
  - Thread per task, can result in threads being exhausted
  - Solution to use a thread pool

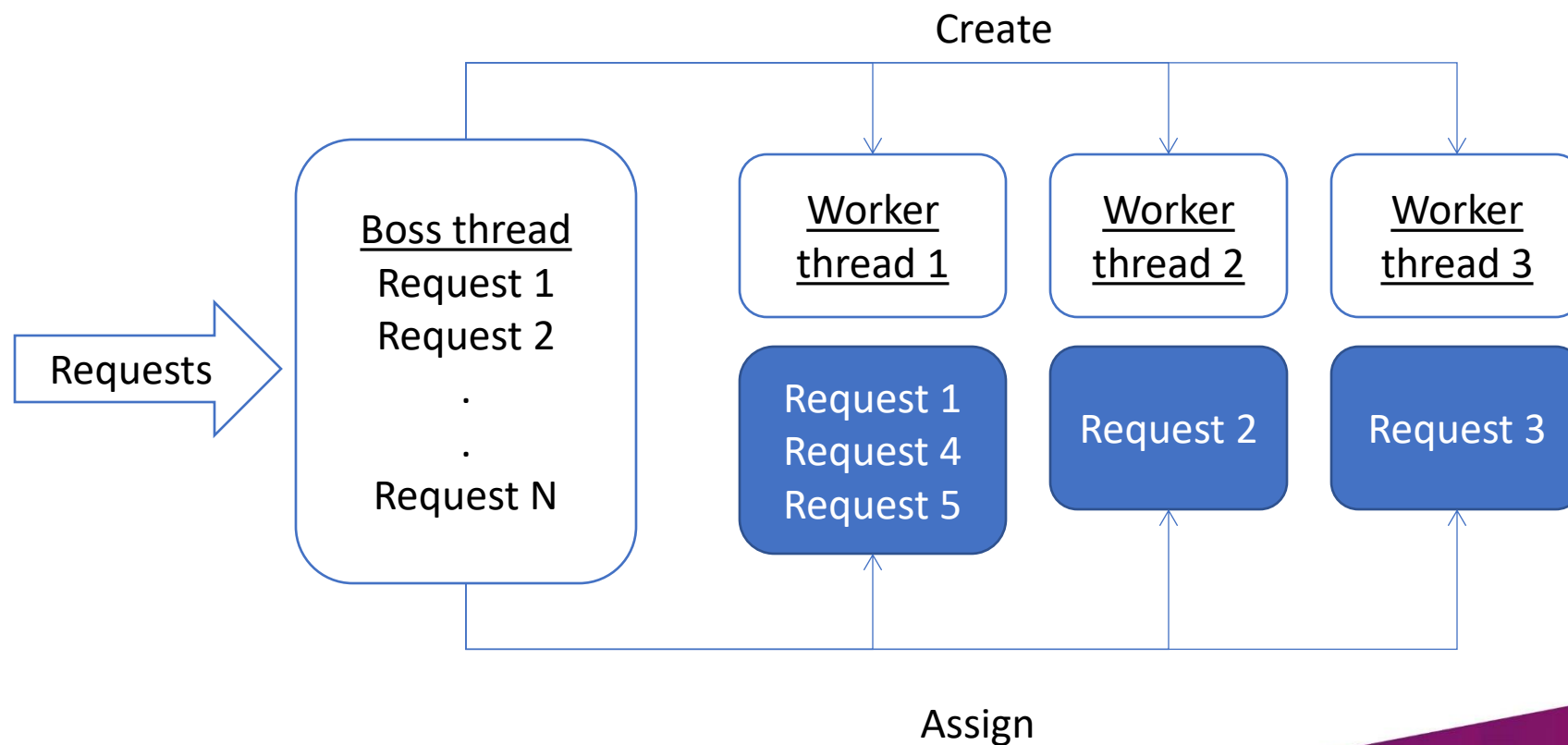
# Delegation – thread per task



# Delegation, contd

- Thread Pool
  - All worker threads start suspended
  - Boss thread wakes thread and assigns a task, this continues until thread pool is exhausted
  - Boss thread queues further tasks
  - Worker thread completes task, then either gets the next task on the queue or suspends

# Delegation – thread pool

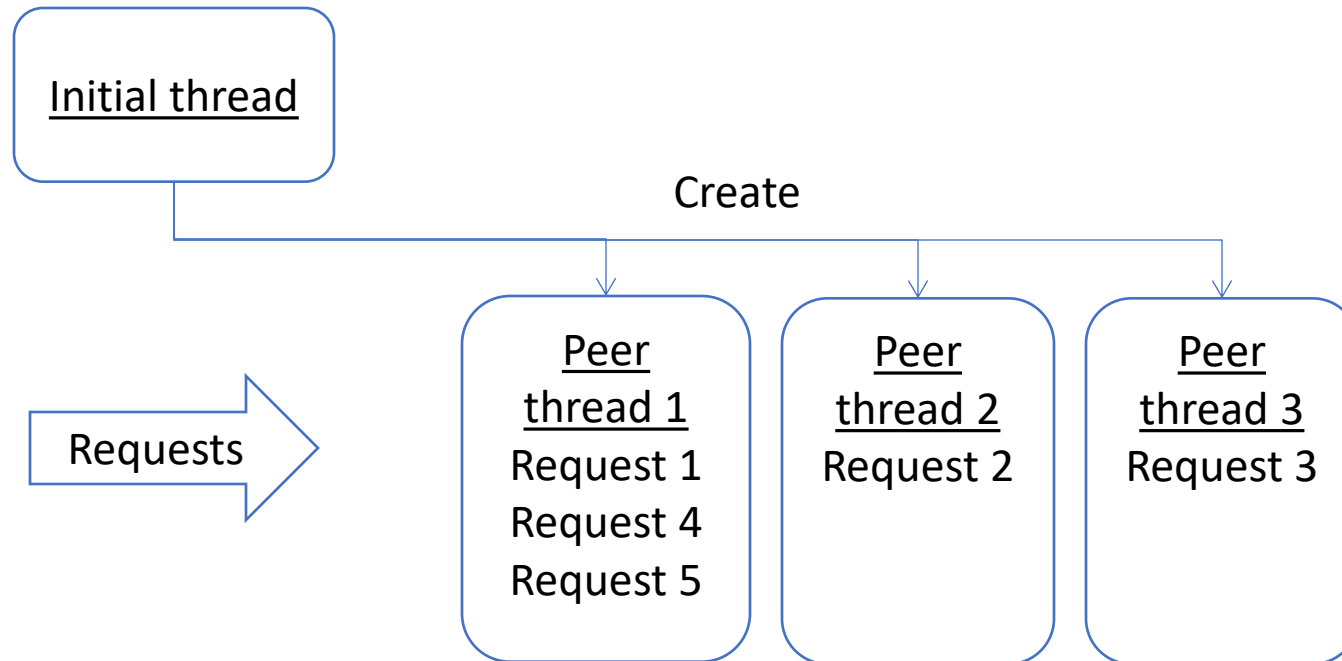


# Peer-to-Peer Model

- Initiation thread
  - Creates peer threads
- Peer thread
  - Select a new task
  - Process a task
  - Synchronise any data



# Peer-to-Peer Model



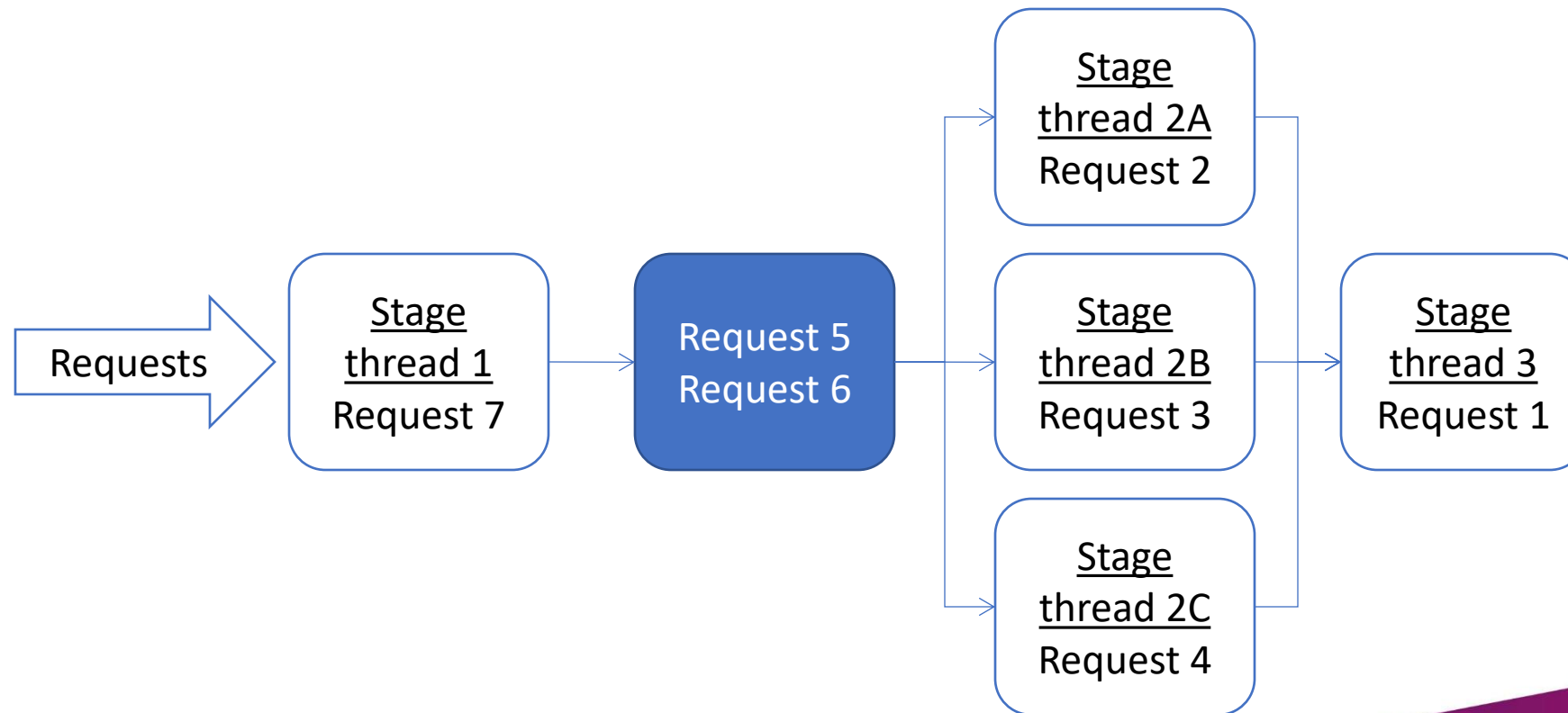
# Pipeline Model

- The pipeline is divided into stages, with each stage being assigned to a separate thread
- Stage thread
  - Takes input from the previous stage
  - Executes a task
  - Sends intermediate or final data to the next stage
- Buffers
  - Added between stages to improve performance

# Pipeline



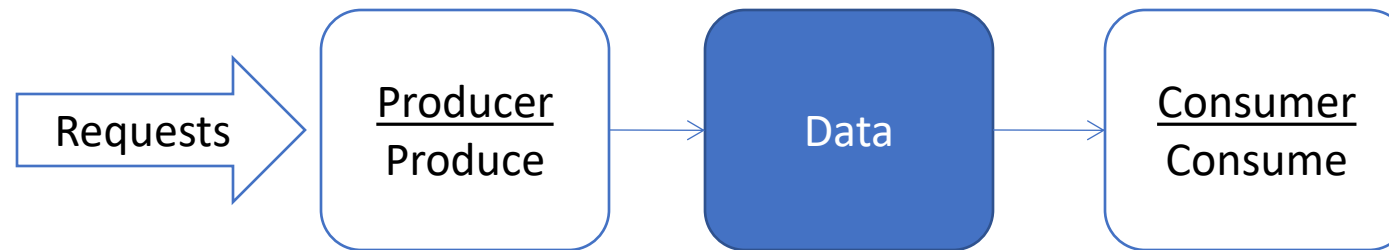
# Pipeline



# Producer-Consumer Model

- Producer thread, that produces data
- Consumer thread, that consumes data
- Data is stored in a buffer
- Placing and retrieving data requires synchronization

# Producer-consumer model



# SPMD and MPMD

- Delegation
  - SPMD and MPMD
- Peer to peer
  - SPMD and MPMD
- Pipeline
  - MPMD only
- Producer Consumer
  - MPMD only



UNIVERSITY  
OF HULL

# Thread pools



# Thread pools

- An alternative approach for deploying threads is to use a thread pool.
- Worker threads in the pool repeatedly retrieve and execute tasks from a task queue.
- Architecturally the approach can produced a very clean design, but it relies on a thread-safe queue.

# Thread pool

```
1.  def worker(threadId, taskQueue):
2.      safePrint('Starting worker', threadId)

3.      task = taskQueue.get()
4.      while task is not None:

5.          safePrint ('Running task',task,'on worker',threadId)
6.          sleep(random())                #simulate work
7.          taskQueue.task_done()
8.          task = taskQueue.get()

9.      taskQueue.task_done()

10. safePrint('Ending worker',threadId)
```

# Thread pool

```
1.  safePrint('Starting main programme')

2.  taskQueue = Queue(3)
3.  threadIds = range(2)
4.  for i in threadIds:
5.      threading.Thread(target = worker, args = [i, taskQueue]).start()

6.  taskIds = range(8)
7.  for i in taskIds:
8.      taskQueue.put(i)      #safePrint('Queued task', i)

9.  for i in threadIds:
10.     taskQueue.put(None)

11. taskQueue.join()  # wait for all work to finish

12. safePrint('Ending main programme')
```

# Thread pool

```
Starting main programme
Starting worker 0
Starting worker 1
Running task 0 on worker 0
Running task 1 on worker 1
Running task 2 on worker 1
Running task 3 on worker 1
Running task 4 on worker 0
Running task 5 on worker 0
Running task 6 on worker 1
Running task 7 on worker 0
Ending worker 1
Ending worker 0
Ending main programme
```

```
Starting main programme
Starting worker 0
Starting worker 1
Queued task 0
Running task 0 on worker 0
Queued task 1
Running task 1 on worker 1
Queued task 2
Queued task 3
Queued task 4
Running task 2 on worker 1
Queued task 5
Running task 3 on worker 0
Queued task 6
Running task 4 on worker 1
Queued task 7
Running task 5 on worker 0
Running task 6 on worker 1
Running task 7 on worker 0
Ending worker 0
Ending worker 1
Ending main programme
```

# Thread pool, the queue

- What synchronisation is required within the Queue data structure?
- How would you implement `queue.task_done()` ?
- How would you implement `queue.join()` ?



UNIVERSITY  
OF HULL

# Hints and tips for threading

# When to use

- When there is a need to schedule work in the background and still update the foreground.
  - E.g. loading a terrain database in the background and rendering in the foreground
- Look for a logical division between threads
- Avoid significant dependencies between threads
  - Results in excessive suspension
- Aim for independent execution
- Two good architectures
  - Always running and independent of first thread. Makes data available to first thread
  - Almost never runs, sleeps, waiting for an event. When the event occurs it runs quickly and then sleeps again

# Warning

- Never underestimate the complexity of creating a multi-threaded application
- Work through all possible combinations of data access
- Design to avoid deadlocks, livelocks and race conditions
- Consider data integrity